Week 4 – Association Analysis

Workflow: **problem → transactions → items → frequent itemsets → association rules → evaluate → filter → interpret**

Tuesday – Concepts

- **Transaction:** one case containing a set of items, e.g. one shopping basket or one student-week.
- **Item / itemset:** something that can occur in a transaction, and a set of items that occur together.
- **Rule A → B:** B often occurs in transactions that contain A. A is the *antecedent* and B is the *consequent*.
- **Support:** the share of all transactions that contain the itemset.
- **Confidence(A → B):** support(A, B) / support(A), i.e. how often B appears when A is present.
- **Lift:** confidence / support(B). A value above 1 means A and B occur together more often than expected, about 1 means no real association, and below 1 means less often than expected.
- **Apriori principle (Zaki & Meira, ch. 8):** if an itemset is not frequent, no larger set that contains it can be frequent, so large parts of the search can be skipped.
- **mlxtend:** `TransactionEncoder` turns lists of items into a True/False table, `apriori()` finds frequent itemsets and `association_rules()` builds rules from them.

Association does **not** mean causation.

Short practical check (small example)

In [1]:
```
# %pip install mlxtend
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

toy = [["bread", "milk"], ["bread", "butter"], ["bread", "milk", "butter"],
       ["milk"], ["bread", "milk"]]
te = TransactionEncoder()
toy_df = pd.DataFrame(te.fit_transform(toy), columns=te.columns_)
display(toy_df)
toy_fi = apriori(toy_df, min_support=0.4, use_colnames=True)
display(toy_fi)
association_rules(toy_fi, metric="confidence", min_threshold=0.5)[["antecedents", "consequents", "support",
"confidence", "lift"]].round(2)
```

	bread	butter	milk
0	True	False	True
1	True	True	False
2	True	True	True
3	False	False	True
4	True	False	True

	support	itemsets
0	0.8	frozenset({bread})
1	0.4	frozenset({butter})
2	0.8	frozenset({milk})
3	0.4	frozenset({butter, bread})
4	0.6	frozenset({milk, bread})

	antecedents	consequents	support	confidence	lift
0	frozenset({butter})	frozenset({bread})	0.4	1.00	1.25
1	frozenset({bread})	frozenset({butter})	0.4	0.50	1.25
2	frozenset({milk})	frozenset({bread})	0.6	0.75	0.94
3	frozenset({bread})	frozenset({milk})	0.6	0.75	0.94

Here one transaction is one shopping basket. For example, the rule bread → milk has support 0.6, confidence 0.75 and lift 0.94, so there is no real association, because milk is common anyway.

Wednesday – Part A: Understand the transactions

In [2]:
```
df = pd.read_csv("synthetic-association-dataset.csv")
print(df.shape)
display(df.head())
print("Transactions:", df["transaction_id"].nunique(), "| Students:", df["student_key"].nunique(),
      "| Weeks:", df["course_week"].nunique(), "| Unique items:", df["item"].nunique())
print("Missing values:", df.isna().sum().sum(), "| Duplicate (transaction, item) rows:",
df.duplicated(["transaction_id", "item"]).sum())
df["item"].value_counts()
```

```
(1992, 5)
```

	transaction_id	student_key	course_week	group_code	item
0	W01-SYN001	SYN001	1	G10	attendance_recorded
1	W01-SYN001	SYN001	1	G10	concept_map_submitted
2	W01-SYN001	SYN001	1	G10	task_submitted
3	W01-SYN001	SYN001	1	G10	task_late
4	W01-SYN001	SYN001	1	G10	quiz_completed

```
Transactions: 320 | Students: 40 | Weeks: 8 | Unique items: 15
Missing values: 0 | Duplicate (transaction, item) rows: 0
item
attendance_recorded          241
task_submitted               215
concept_map_submitted        212
quiz_completed               209
practical_verified           194
project_activity             189
task_on_time                 146
asked_for_help               118
multiple_attempts            105
high_monday_activity         104
quiz_high                     84
task_late                     69
low_monday_activity           64
quiz_low                      40
minimal_recorded_activity      2
Name: count, dtype: int64
```

- **One transaction** = one student in one course week (`transaction_id`, e.g. W01-SYN001). There are 320 transactions (40 students × 8 weeks).
- **Items:** 15 recorded events or conditions, such as `task_submitted`, `quiz_high` and `asked_for_help`.
- **Identifiers, not items:** `transaction_id`, `student_key`, `course_week`, `group_code`.
- **Duplicates / inconsistencies:** there are no duplicate rows. Some items are **derived from others**: `task_on_time` and `task_late` always come with `task_submitted`, and `quiz_high` and `quiz_low` always come with `quiz_completed`. This creates obvious rules (see Thursday).
- **Missing values:** none.
- **Meaningful?** Mostly yes. `minimal_recorded_activity` occurs only twice, which is too rare to be useful. `quiz_high` and `quiz_low` depend on a chosen score threshold.

Part B: Prepare transaction data

In [3]:
```
transactions = df.groupby("transaction_id")["item"].apply(list)
print(transactions.head(3))

te = TransactionEncoder()
basket = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_, index=transactions.index)
print(basket.shape)
basket.head()
```

```
transaction_id
W01-SYN001    [attendance_recorded, concept_map_submitted, t...
W01-SYN002    [attendance_recorded, concept_map_submitted, t...
W01-SYN003                                    [project_activity]
Name: item, dtype: object
(320, 15)
```

	asked_for_help	attendance_recorded	concept_map_submitted	high_monday_activity	low_monday_activity	minimal_recorded_activity	multiple_attempts	practical_verified	project_activity	quiz_completed	quiz_high	quiz_low	task_late	task_on_time	task_submitted
transaction_id															
W01-SYN001	True	True	True	True	False	False	False	True	True	True	False	False	True	False	True
W01-SYN002	False	True	True	False	True	False	False	True	True	True	False	False	False	True	True
W01-SYN003	False	False	False	False	False	False	False	False	True	False	False	False	False	False	False
W01-SYN004	False	False	True	False	False	False	False	True	False	True	True	False	False	True	True
W01-SYN005	True	True	True	False	False	False	True	True	True	True	False	False	False	True	True

`True` in an item column means that the event was **recorded** for that student in that week. `False` means it was not recorded. That is not always the same as "did not happen", because Edge-LMS only observes some activity.

Part C: Frequent itemsets

In [4]:
```
fi_10 = apriori(basket, min_support=0.10, use_colnames=True)
fi_10["length"] = fi_10["itemsets"].apply(len)
print("Frequent itemsets (min_support=0.10):", len(fi_10))
print(fi_10["length"].value_counts().sort_index())
print("\nMost common single items:")
display(fi_10[fi_10["length"] == 1].sort_values("support", ascending=False).head(6))
print("Most common combinations:")
fi_10[fi_10["length"] >= 2].sort_values("support", ascending=False).head(6)
```

```
Frequent itemsets (min_support=0.10): 467
length
1     14
2     67
3    139
4    150
5     78
6     18
7      1
Name: count, dtype: int64

Most common single items:
```

	support	itemsets	length
1	0.753125	frozenset({attendance_recorded})	1
13	0.671875	frozenset({task_submitted})	1
2	0.662500	frozenset({concept_map_submitted})	1
8	0.653125	frozenset({quiz_completed})	1
6	0.606250	frozenset({practical_verified})	1
7	0.590625	frozenset({project_activity})	1

```
Most common combinations:
```

	support	itemsets	length
30	0.559375	frozenset({quiz_completed, attendance_recorded})	2
34	0.531250	frozenset({task_submitted, attendance_recorded})	2
29	0.512500	frozenset({project_activity, attendance_record...	2
24	0.509375	frozenset({concept_map_submitted, attendance_r...	2
28	0.506250	frozenset({practical_verified, attendance_reco...	2
66	0.484375	frozenset({practical_verified, task_submitted})	2

In [5]:
```
for s in [0.30, 0.20, 0.10, 0.05]:
    print(f"min_support={s:.2f}: {len(apriori(basket, min_support=s, use_colnames=True))} frequent itemsets")
```

```
min_support=0.30: 52 frequent itemsets
min_support=0.20: 132 frequent itemsets
min_support=0.10: 467 frequent itemsets
min_support=0.05: 1131 frequent itemsets
```

When minimum support is lowered, the number of frequent itemsets grows fast (from 52 to 1,131), because many rarer and longer combinations now pass the threshold.

Part D: Association rules

In [6]:
```
rules = association_rules(fi_10, metric="confidence", min_threshold=0.6)
print("Rules (support ≥ 0.10, confidence ≥ 0.6):", len(rules))

# Smaller table: one item → one item, lift ≥ 1.2, and without the rules that are true by construction
trivial = {("task_on_time", "task_submitted"), ("task_late", "task_submitted"),
           ("quiz_high", "quiz_completed"), ("quiz_low", "quiz_completed"),
           ("high_monday_activity", "attendance_recorded"), ("low_monday_activity", "attendance_recorded")}
all_rules = association_rules(fi_10, metric="lift", min_threshold=0)
simple = all_rules[(all_rules["antecedents"].apply(len) == 1) & (all_rules["consequents"].apply(len) == 1)].copy()
simple["A"] = simple["antecedents"].apply(lambda s: next(iter(s)))
simple["B"] = simple["consequents"].apply(lambda s: next(iter(s)))
simple = simple[~simple.apply(lambda r: (r["A"], r["B"]) in trivial or (r["B"], r["A"]) in trivial, axis=1)]
interesting = simple[simple["lift"] >= 1.2].sort_values("lift", ascending=False)
interesting[["A", "B", "support", "confidence", "lift"]].round(3)
```

```
Rules (support ≥ 0.10, confidence ≥ 0.6): 2033
```

	A	B	support	confidence	lift
71	high_monday_activity	quiz_high	0.122	0.375	1.429
70	quiz_high	high_monday_activity	0.122	0.464	1.429
7	low_monday_activity	asked_for_help	0.100	0.500	1.356
6	asked_for_help	low_monday_activity	0.100	0.271	1.356
126	quiz_high	task_on_time	0.153	0.583	1.279
127	task_on_time	quiz_high	0.153	0.336	1.279
100	practical_verified	task_late	0.166	0.273	1.267
101	task_late	practical_verified	0.166	0.768	1.267
72	task_on_time	high_monday_activity	0.188	0.411	1.264
73	high_monday_activity	task_on_time	0.188	0.577	1.264
99	quiz_high	practical_verified	0.194	0.738	1.217
98	practical_verified	quiz_high	0.194	0.320	1.217

Part E: Interpret three rules

	Rule 1: `quiz_high → task_on_time`	**Rule 2: `low_monday_activity → asked_for_help`**	**Rule 3: `high_monday_activity → quiz_high`**
Antecedent	quiz score was in the high band that week	low recorded Monday activity	high recorded Monday activity
Consequent	task was submitted on time	the student asked for help	quiz score in the high band
Support	0.153	0.100	0.122
Confidence	0.583	0.500	0.375
Lift	1.28	1.36	1.43
Reasonable conclusion	a high quiz score and on-time submission occur together more often than expected	low recorded activity and asking for help occur together more often than expected	these two occur together about 1.4× more often than expected, but only in about a third of high-activity weeks
Cannot conclude	that a high quiz score *causes* on-time submission, or the other way round	that low activity causes asking for help, or that these students are "weak"	that Monday activity raises quiz scores

All three rules describe **co-occurrence in this synthetic dataset**, not cause and effect.

Thursday – 1. Rule assessment

Confidence can be high just because the consequent is common. For example, `attendance_recorded` appears in 75 % of all transactions, so almost any rule → attendance has high confidence. Lift corrects for this by dividing by the support of the consequent.

2. Threshold sensitivity

In [7]:
```
rows = []
for name, s, c, l in [("A", 0.30, 0.6, 0), ("B", 0.20, 0.6, 0), ("C", 0.10, 0.6, 0),
                      ("D", 0.05, 0.6, 0), ("E", 0.10, 0.8, 0), ("F", 0.10, 0.6, 1.0)]:
    fi = apriori(basket, min_support=s, use_colnames=True)
    r = association_rules(fi, metric="confidence", min_threshold=c)
    r = r[r["lift"] > l] if l else r
    rows.append({"setting": name, "min_support": s, "min_conf": c, "min_lift": l or "-",
                 "itemsets": len(fi), "rules": len(r)})
pd.DataFrame(rows)
```

	setting	min_support	min_conf	min_lift	itemsets	rules
0	A	0.30	0.6	-	52	114
1	B	0.20	0.6	-	132	406
2	C	0.10	0.6	-	467	2033
3	D	0.05	0.6	-	1131	6739
4	E	0.10	0.8	-	467	573
5	F	0.10	0.6	1.0	467	2003

Setting	Main observation
A (support 0.30)	few rules, mostly between very common items such as attendance, task submitted and concept map
C (support 0.10)	about 2,000 rules, many of them repeating the same pattern with extra items
D (support 0.05)	about 6,700 rules. Rare patterns appear, based on only around 16 transactions
E (confidence 0.8)	far fewer rules. Many remaining rules are "→ attendance_recorded" or rules that are true by construction
F (lift > 1)	removes 30 rules whose consequent is simply common (lift ≤ 1)

- **Lowering support:** many more itemsets and rules, including very rare ones.
- **Raising confidence:** fewer rules, but still not necessarily more interesting ones.
- **Requiring lift > 1:** rules such as `task_late → attendance_recorded` (lift 0.98) disappear.
- **Statistically interesting but practically unimportant:** yes. For example, `quiz_high → quiz_completed` has lift 1.53 but is obvious, because a quiz score needs a completed quiz.

The thresholds are my choices, not natural truths in the data.

3. Investigate a misleading rule

In [8]:
```
check = all_rules[(all_rules["antecedents"] == frozenset({"concept_map_submitted"})) &
                  (all_rules["consequents"] == frozenset({"attendance_recorded"}))]
display(check[["antecedents", "consequents", "support", "confidence", "lift"]].round(3))
print("Support of attendance_recorded alone:", round(basket["attendance_recorded"].mean(), 3))
```

	antecedents	consequents	support	confidence	lift
21	frozenset({concept_map_submitted})	frozenset({attendance_recorded})	0.509	0.769	1.021

```
Support of attendance_recorded alone: 0.753
```

Rule: `concept_map_submitted → attendance_recorded`

Why it looked interesting: high support (0.51) and confidence 0.77, so it seems that students who submit the concept map also attend.

Additional evidence: attendance is recorded in **75 %** of all transactions anyway, so the lift is only **1.02**.

Revised interpretation: there is almost no association. The high confidence only comes from attendance being very common. A similar case is `quiz_high → quiz_completed` (confidence 1.0), which is true just because of how the items were built.

4. Interpretation, validity and ethics

1. **Enough transactions for rare patterns?** No. With 320 transactions, a support of 0.05 is only about 16 cases, which is too few to trust.
2. **Independent observations?** No. Each student appears in 8 transactions, so one student's habits can create a pattern.
3. **Item construction?** Yes. Derived items (on time → submitted, quiz high → completed, activity → attendance) create rules automatically.
4. **Third variable?** Possibly. For example, general engagement or the course week could explain both a high quiz score and on-time submission.
5. **Does the data observe real behaviour?** Only what Edge-LMS records, mainly during Monday classes. A missing item does not mean no study activity.
6. **Unfair labels?** Yes. Rules like low activity → asked for help could be misused to label students or groups.
7. **Identifiable individuals?** With only 40 students, rare combinations of items could point to a specific person.
8. **Before a real decision:** more and real data, checks across different weeks and groups, discussion with teachers, and a clear, fair purpose.

Safe wording: "*These recorded events frequently occurred together.*"

Friday – Practical verification values

In [9]:
```
print("Transactions:", basket.shape[0])
print("Unique items:", basket.shape[1])
print("Selected min_support: 0.10")
print("Frequent itemsets:", len(fi_10))
print("Rules (confidence ≥ 0.6):", len(rules))
```

```
Transactions: 320
Unique items: 15
Selected min_support: 0.10
Frequent itemsets: 467
Rules (confidence ≥ 0.6): 2033
```

Friday – Concept map

